

```

options(OutDec = ".") # Asegurar punto decimal en este chunk

# Establecer semilla aleatoria
set.seed(sample(1:10000, 1))

# Aleatorizar dimensiones del parabrisas
largo_base <- sample(10:15, 1) # Largo base entre 10 y 15 cm
ancho_base <- sample(6:10, 1)  # Ancho base entre 6 y 10 cm

# Aleatorizar longitud de las plumillas
longitud_plumilla_base <- sample(4:8, 1) # Longitud base entre 4 y 8 cm

# Aleatorizar términos para el contexto del problema
objetos <- c("parabrisas", "ventana frontal", "vidrio delantero", "cristal frontal")
objeto <- sample(objetos, 1)

vehiculos <- c("carrito de juguete", "coche en miniatura", "automóvil de juguete",
              "vehículo de juguete", "modelo a escala")
vehiculo <- sample(vehiculos, 1)

terminos_limpiador <- c("plumilla", "limpiador", "limpiaparabrisas", "escobilla")
termino_limpiador <- sample(terminos_limpiador, 1)

formas <- c("rectangular", "de forma rectangular", "con forma de rectángulo")
forma <- sample(formas, 1)

angulos <- c("90°", "90 grados", "ángulo recto")
angulo <- sample(angulos, 1)

# Aleatorizar la opción correcta
opciones <- c("A", "B", "C", "D")
opcion_correcta <- sample(opciones, 1)
indice_correcto <- which(opciones == opcion_correcta)

# Vector de solución para r-exams
solucion <- integer(4)
solucion[indice_correcto] <- 1

# Configurar opciones con variaciones específicas para cada opción
# Las dimensiones de la opción correcta
if (opcion_correcta == "A") {
  # Si A es la correcta
  opcion_A <- list(
    largo = largo_base,
    ancho = ancho_base,
    plumilla1 = longitud_plumilla_base,
    plumilla2 = longitud_plumilla_base
  )

  # Opciones incorrectas
  opcion_B <- list(
    largo = largo_base,
    ancho = ancho_base,

```

```

    plumilla1 = longitud_plumilla_base,
    plumilla2 = longitud_plumilla_base + sample(1:3, 1)
  )

opcion_C <- list(
  largo = largo_base,
  ancho = ancho_base,
  plumilla1 = longitud_plumilla_base - sample(1:2, 1),
  plumilla2 = longitud_plumilla_base - sample(1:2, 1)
)

opcion_D <- list(
  largo = largo_base,
  ancho = ancho_base - sample(1:2, 1),
  plumilla1 = longitud_plumilla_base + sample(1:3, 1),
  plumilla2 = longitud_plumilla_base + sample(1:3, 1)
)
} else if (opcion_correcta == "B") {
  # Si B es la correcta
  opcion_B <- list(
    largo = largo_base,
    ancho = ancho_base,
    plumilla1 = longitud_plumilla_base,
    plumilla2 = longitud_plumilla_base + sample(1:3, 1)
  )

  # Opciones incorrectas
  opcion_A <- list(
    largo = largo_base,
    ancho = ancho_base,
    plumilla1 = longitud_plumilla_base,
    plumilla2 = longitud_plumilla_base
  )

  opcion_C <- list(
    largo = largo_base,
    ancho = ancho_base,
    plumilla1 = longitud_plumilla_base - sample(1:2, 1),
    plumilla2 = longitud_plumilla_base - sample(1:2, 1)
  )

  opcion_D <- list(
    largo = largo_base,
    ancho = ancho_base - sample(1:2, 1),
    plumilla1 = longitud_plumilla_base + sample(1:3, 1),
    plumilla2 = longitud_plumilla_base + sample(1:3, 1)
  )
} else if (opcion_correcta == "C") {
  # Si C es la correcta
  opcion_C <- list(
    largo = largo_base,
    ancho = ancho_base,
    plumilla1 = longitud_plumilla_base,

```

```

    plumilla2 = longitud_plumilla_base
  )

  # Opciones incorrectas
  opcion_A <- list(
    largo = largo_base - sample(1:2, 1),
    ancho = ancho_base,
    plumilla1 = longitud_plumilla_base,
    plumilla2 = longitud_plumilla_base
  )

  opcion_B <- list(
    largo = largo_base,
    ancho = ancho_base,
    plumilla1 = longitud_plumilla_base,
    plumilla2 = longitud_plumilla_base + sample(1:3, 1)
  )

  opcion_D <- list(
    largo = largo_base,
    ancho = ancho_base - sample(1:2, 1),
    plumilla1 = longitud_plumilla_base + sample(1:3, 1),
    plumilla2 = longitud_plumilla_base + sample(1:3, 1)
  )
} else if (opcion_correcta == "D") {
  # Si D es la correcta
  opcion_D <- list(
    largo = largo_base,
    ancho = ancho_base - sample(1:2, 1),
    plumilla1 = longitud_plumilla_base + sample(1:3, 1),
    plumilla2 = longitud_plumilla_base + sample(1:3, 1)
  )

  # Opciones incorrectas
  opcion_A <- list(
    largo = largo_base,
    ancho = ancho_base,
    plumilla1 = longitud_plumilla_base,
    plumilla2 = longitud_plumilla_base
  )

  opcion_B <- list(
    largo = largo_base,
    ancho = ancho_base,
    plumilla1 = longitud_plumilla_base,
    plumilla2 = longitud_plumilla_base + sample(1:3, 1)
  )

  opcion_C <- list(
    largo = largo_base,
    ancho = ancho_base,
    plumilla1 = longitud_plumilla_base,
    plumilla2 = longitud_plumilla_base
  )
}

```

```

)
}

# Las dimensiones de la respuesta correcta según la opción elegida
largo <- get(paste0("opcion_", opcion_correcta))$largo
ancho <- get(paste0("opcion_", opcion_correcta))$ancho
longitud_plumilla1 <- get(paste0("opcion_", opcion_correcta))$plumilla1
longitud_plumilla2 <- get(paste0("opcion_", opcion_correcta))$plumilla2

# Para mostrar en el enunciado (podría ser diferente según la opción correcta)
largo_texto <- largo
ancho_texto <- ancho

# Si las plumillas son iguales en la opción correcta, mostrarlo así
if (longitud_plumilla1 == longitud_plumilla2) {
  longitud_plumilla_texto <- longitud_plumilla1
  texto_longitud <- paste0(longitud_plumilla_texto, " cm de longitud cada una")
} else {
  longitud_plumilla_texto <- paste0(longitud_plumilla1, " cm y ", longitud_plumilla2, " cm")
  texto_longitud <- paste0("longitudes de ", longitud_plumilla_texto, " respectivamente")
}

options(OutDec = ".") # Asegurar punto decimal en este chunk

# Función para generar código Python para dibujar el esquema
generar_esquema_python <- function(opcion, letra) {
  largo <- opcion$largo
  ancho <- opcion$ancho
  plumilla1 <- opcion$plumilla1
  plumilla2 <- opcion$plumilla2

  codigo <- paste0('
import matplotlib.pyplot as plt
import numpy as np

fig, ax = plt.subplots(figsize=(8, 4))

# Rectángulo punteado
rect_x = [0.5, 6.7, 6.7, 0.5, 0.5]
rect_y = [0, 0, 3, 3, 0]
ax.plot(rect_x, rect_y, linestyle="dashed", color="gray", dashes=(4, 4))

def dibujar_abanico(x_origin, y_origin, radius, lines_count=5):
  # Arco sólido
  theta = np.linspace(0, np.pi/2, 100)
  x_arc = x_origin + radius * np.cos(theta)
  y_arc = y_origin + radius * np.sin(theta)
  ax.plot(x_arc, y_arc, color="black", linestyle="solid")

  # Líneas radiales
  for i in range(lines_count):
    angle = (np.pi/2) * i / (lines_count - 1)
    x_end = x_origin + radius * np.cos(angle)
    y_end = y_origin + radius * np.sin(angle)

```

```

    # Todas las líneas sólidas
    if i == 0 or i == lines_count-1:
        linewidth = 1.5
        ax.plot([x_origin, x_end], [y_origin, y_end], color="black", linewidth=linewidth)
    else:
        pass

    # Añadir símbolo de 90 grados en el vértice
    square_size = 0.15
    x_square = [x_origin, x_origin + square_size, x_origin + square_size, x_origin, x_origin]
    y_square = [y_origin, y_origin, y_origin + square_size, y_origin + square_size, y_origin]
    ax.plot(x_square, y_square, color="black", linewidth=0.8)

# Posición de los abanicos
dibujar_abanico(1, 0, 2.5, lines_count=2)
dibujar_abanico(3.75, 0, 2.5, lines_count=2)

# Añadir las dimensiones
# Dimensión horizontal superior (largo)
ax.annotate('', largo, ' cm', xy=(3.6, 3.2), xytext=(3.6, 3.2), ha="center", color="red")
ax.plot([0.5, 6.7], [3.1, 3.1], color="red", linewidth=1)
ax.plot([0.5, 0.5], [3.05, 3.15], color="red", linewidth=1)
ax.plot([6.7, 6.7], [3.05, 3.15], color="red", linewidth=1)

# Dimensión vertical (ancho)
ax.annotate('', ancho, ' cm', xy=(0, 1.5), xytext=(0.15, 1.5), rotation=90, va="center", color="red")
ax.plot([0.4, 0.4], [0, 3], color="red", linewidth=1)
ax.plot([0.35, 0.45], [0, 0], color="red", linewidth=1)
ax.plot([0.35, 0.45], [3, 3], color="red", linewidth=1)

# Dimensiones horizontales inferiores (plumillas)
ax.annotate('', plumilla1, ' cm', xy=(1.75, -0.3), xytext=(2.3, -0.45), ha="center", color="red")
ax.plot([1, 3.5], [-0.2, -0.2], color="red", linewidth=1)
ax.plot([1, 1], [-0.25, -0.15], color="red", linewidth=1)
ax.plot([3.5, 3.5], [-0.25, -0.15], color="red", linewidth=1)

ax.annotate('', plumilla2, ' cm', xy=(4.5, -0.3), xytext=(5, -0.45), ha="center", color="red")
ax.plot([3.75, 6.25], [-0.2, -0.2], color="red", linewidth=1)
ax.plot([3.75, 3.75], [-0.25, -0.15], color="red", linewidth=1)
ax.plot([6.25, 6.25], [-0.25, -0.15], color="red", linewidth=1)

# Ajustes de gráficos
ax.set_aspect("equal")
ax.axis("off")
plt.xlim(0, 7.5)
plt.ylim(-0.6, 3.5)

plt.tight_layout()
plt.savefig("opcion", letra, '.png', dpi=150, bbox_inches="tight")
plt.close()
')

return(codigo)

```

```

}

# Generar código Python para cada opción
codigo_opcion_A <- generar_esquema_python(opcion_A, "A")
codigo_opcion_B <- generar_esquema_python(opcion_B, "B")
codigo_opcion_C <- generar_esquema_python(opcion_C, "C")
codigo_opcion_D <- generar_esquema_python(opcion_D, "D")

# Ejecutar código Python para generar las figuras
py_run_string(codigo_opcion_A)
py_run_string(codigo_opcion_B)
py_run_string(codigo_opcion_C)
py_run_string(codigo_opcion_D)

options(OutDec = ".") # Asegurar punto decimal en este chunk

# Código Python para generar el diagrama de plumillas con ángulo
codigo_diagrama_plumilla <- '
import matplotlib.pyplot as plt
import numpy as np

fig, ax = plt.subplots(figsize=(8, 4))

# Rectángulo punteado
rect_x = [0.5, 6.7, 6.7, 0.5, 0.5]
rect_y = [0, 0, 3, 3, 0]
ax.plot(rect_x, rect_y, linestyle="dashed", color="gray", dashes=(4, 4))

def dibujar_abanico(x_origin, y_origin, radius, lines_count=5):
    # Arco punteado
    theta = np.linspace(0, np.pi/2, 100)
    x_arc = x_origin + radius * np.cos(theta)
    y_arc = y_origin + radius * np.sin(theta)
    ax.plot(x_arc, y_arc, color="gray", linestyle="dashed", dashes=(4,4))

    # Líneas radiales
    for i in range(lines_count):
        angle = (np.pi/2) * i / (lines_count - 1)
        x_end = x_origin + radius * np.cos(angle)
        y_end = y_origin + radius * np.sin(angle)

        if i == 0 or i == 3:
            linewidth = 2.0 if i == 0 else 1.5
            ax.plot([x_origin, x_end], [y_origin, y_end], color="black", linewidth=linewidth)
        else:
            ax.plot([x_origin, x_end], [y_origin, y_end],
                    color="gray", linestyle="dashed", dashes=(4,4), linewidth=0.8)

# Posición de los abanicos
dibujar_abanico(1, 0, 2.5, lines_count=5)
dibujar_abanico(3.75, 0, 2.5, lines_count=5)

# Ajustar la posición de la flecha y texto "Plumilla"
ax.annotate("Plumilla", xy=(1.7, 1.7), xytext=(0.5, 1.5),

```

```

        arrowprops=dict(facecolor="black", arrowstyle="->", linewidth=1),
        fontsize=10, ha="right", va="center")

# Ajustes de gráficos
ax.set_aspect("equal")
ax.axis("off")
plt.xlim(0, 7.5)
plt.ylim(-0.6, 3.5)

plt.tight_layout()
plt.savefig("diagrama_plumilla.png", dpi=150, bbox_inches="tight")
plt.close()
'

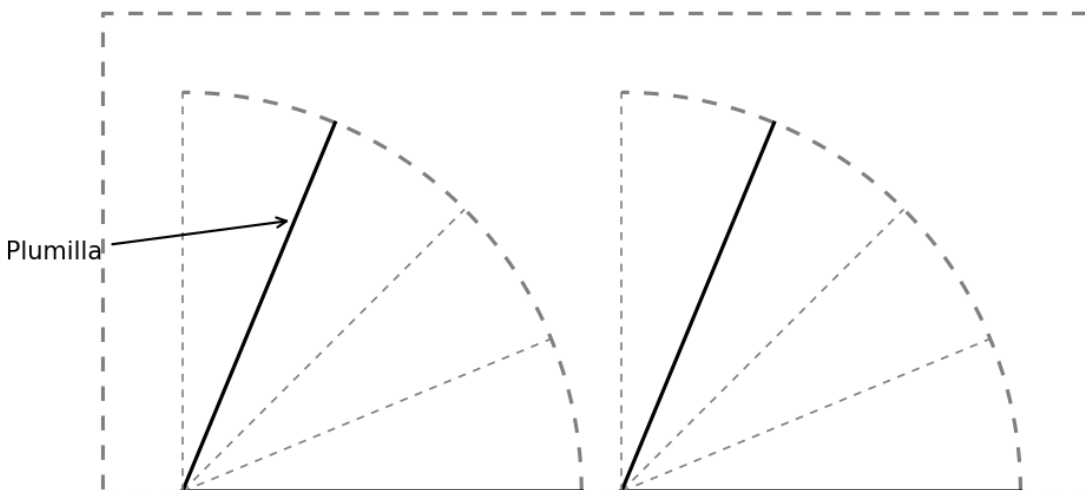
# Ejecutar código Python para generar el diagrama de plumillas
py_run_string(codigo_diagrama_plumilla)

```

Question

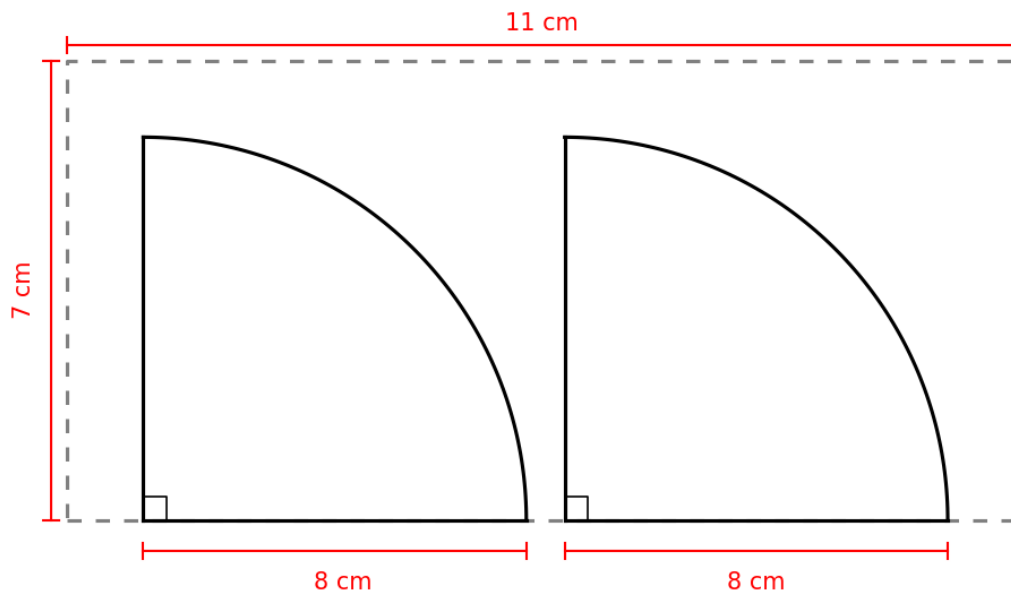
Las dimensiones de un ventana frontal son 11 cm de largo y 7 cm de ancho. Este es un ventana frontal plano y rectangular, correspondiente a un carrito de juguete.

El carrito de juguete cuenta con 2 plumillas con 8 cm de longitud cada una para limpiar el ventana frontal. Estas tienen un ángulo de apertura de 90° , tal como se muestra a continuación.

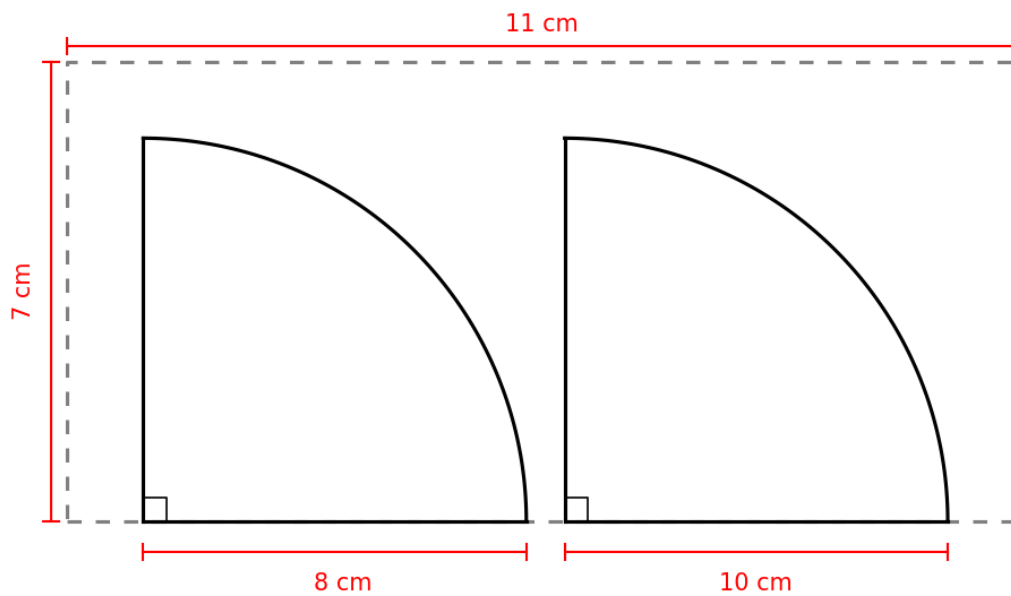


El esquema que representa los datos de las dimensiones del ventana frontal es:

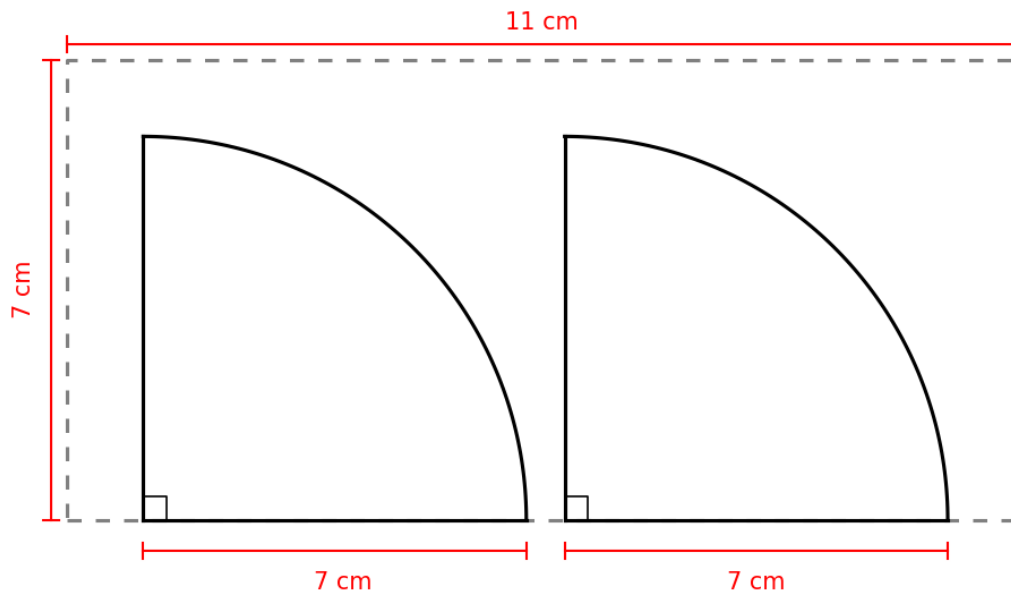
Answerlist



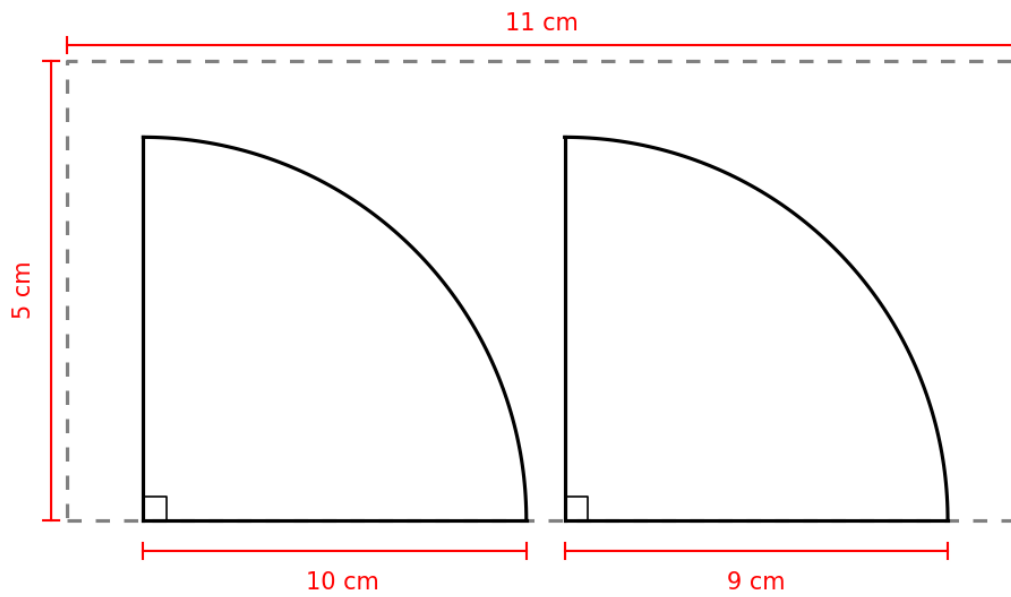
•



•



•



•

Solution

El esquema correcto es la opción A.

Para verificar cuál es el esquema correcto, debemos comparar las especificaciones dadas con cada una de las opciones:

- El ventana frontal tiene 11 cm de largo y 7 cm de ancho.
- Cuenta con 2 plumillas con 8 cm de longitud cada una.
- Las plumillas tienen un ángulo de apertura de 90° .

La opción A muestra correctamente:

- Un rectángulo de 11 cm $\times$ 7 cm.
- Dos plumillas con longitud de 8 cm cada una.
- Ambas plumillas tienen un ángulo de barrido de 90°.

Las otras opciones presentan inconsistencias con los datos proporcionados:

- Opción B: Las plumillas miden 8 cm y 10 cm en lugar de 8 cm cada una.
- Opción C: Las plumillas miden 7 cm y 7 cm en lugar de 8 cm cada una.
- Opción D: El ancho es 5 cm en lugar de 7 cm; Las plumillas miden 10 cm y 9 cm en lugar de 8 cm cada una.

Answerlist

- Verdadero
- Falso
- Falso
- Falso

Meta-information

exname: parabrisas__plumillas__juguete extype: schoice exsolution: 1000 exshuffle: TRUE exsection: Geometría